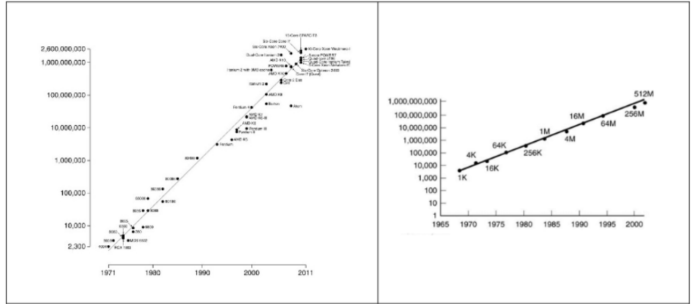


LEGGE DI MOORE:

GORDON MOORE, COFONDATORE DI INTEL, FORMULÒ UNA LEGGE CHIAMATA "LEGGE DI MOORE", DOVE IPOTIZZÒ CHE OGNI SALTO GENERAZIONALE, OVERO OGNI 3 ANNI IL NUMERO DI TRANSISTOR NEI CIRCUITI INTEGRATI SAREBBE QUADRUPLICATO (O RADDOPPIATO OGNI 18-24 MESI), TUTTAVIA L'ANDAMENTO RALLENTÒ DOPO AVER PRATICAMENTE RAGGIUNTO IL LIMITE FISICO DEI TRANSISTOR.

Es 1.
Si considerino i due grafici seguenti, che si riferiscono alla legge di Moore applicata ai chip delle CPU e delle memorie. La legge di Moore è più una previsione derivante da un'osservazione che una legge vera e propria: (a) riportate (con parole vostre) l'enunciato di questa "legge". (b) Quali misure sono rappresentate sull'asse delle ascisse e sull'asse delle ordinate nei due grafici? (c) La scala sull'asse delle ordinate è logaritmica: cosa significa e perché si usa questa scala in questi grafici?

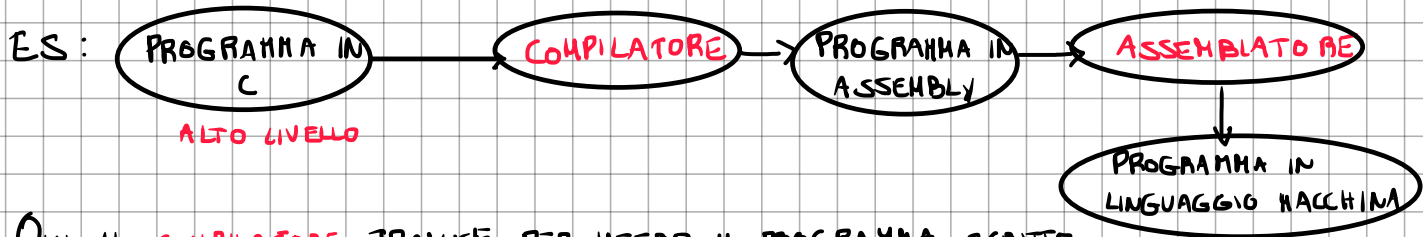


QUESTI SONO 2 GRAFICI CHE RAPPRESENTANO CIÒ CHE DESCRIVE MOORE NELLA SUA LEGGE, NEL GRAFICO A SINISTRA SULLE ASCISSE ABBIAMO IL TEMPO IN ANNI E SULLE ORDINATE IL NUMERO DI TRANSISTOR, NEL GRAFICO A DESTRA ABBIAMO IL TEMPO IN ANNI E SULLE ORDINATE LA CAPACITÀ DELLE MEMORIE.

L'AUMENTO DI TRANSISTOR HA PERMESSO LA REALIZZAZIONE DI MEMORIA CACHE, PIPELINE E MULTICORE

QUALI SONO I PRINCIPI DI PROGETTAZIONE RICORRENTI?

- DEFINIRE DIVERSI LIVELLI DI ASTRAZIONE, OVERO DIVIDERE IN SOTTOPROBLEMI CON UNA STRUTTURA A LIVELLI:



QUI IL COMPILATORE TRADUCE PER INTERO IL PROGRAMMA SCRITTO IN LINGUAGGIO DI ALTO LIVELLO IN LINGUAGGIO MACCHINA (INCORPORANDO L'ASSEMBLATORE). INVECE L'INTERPRETE SCRITTO GIÀ IN LINGUAGGIO MACCHINA PRELEVA, DECODIFICA ED ESEGUE DIRETTAMENTE IL CODICE DEL PROGRAMMA IN LINGUAGGIO DI ALTO LIVELLO.

QUINDI IL COMPILATORE È + VELOCE PERCHÉ NON DEVE AD OGNI CICLO PRELEVARE, DECODIFICARE ED ESEGUIRE MA LO FA UNA VOLTA SOLA E FACENDOLO DIRETTAMENTE SU TUTTO IL PROGRAMMA HA UNO SCOPE + AMPIO E PUÒ OTTIMIZZARE BENE LE ISTRUZIONI, MA IL CODICE NON PUÒ ESSERE ESEGUITO SU TUTTI I CALCOLATORI, HA SOLO SU QUELLO IN CUI È STATO COMPILATO.

MENTRE L'INTERPRETE PUR ESSENDO + LENTO, PERMETTE DI ESEGUIRE LO STESSO V SORGENTE OVUNQUE (BASTA CHE SIA INSTALLATO L'INTERPRETE).

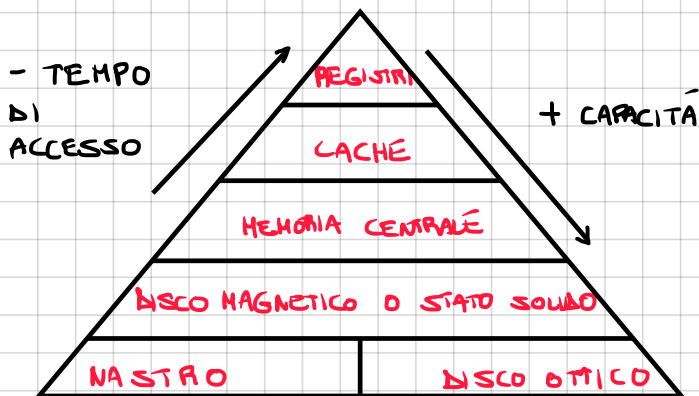
COME FUNZIONA JAVA?



- CERCARE DI MIGLIORARE LE PRESTAZIONI:

OTTIMIZZANDO LE FUNZIONI PIÙ USATE, SFRUTTARE IL PARALLELISMO, SUDDIVIDERE LE FASI DA ESEGUIRE IN PIPELINE E ANTICIPARE FASI GRAZIE ALLA PREVISIONE.

- GESTIRE IN MODO EFFICIENTE I DIVERSI TIPI DI MEMORIA.



REGISTRI = PICCOLE MEMORIE VELOCISSIME NELLA CPU USATE PER ESEGUIRE OPERAZIONI CON LA ALU E SALVARE TEMPORANEAMENTE RISULTATI.

CACHE = MEMORIA COLLOCATA LOGICAMENTE TRA CPU E RAM, CREATA CON SRAM, IL CUI COMPITO È QUELLO DI RENDERE DISPONIBILI DEI DATI ALLA CPU IN MODO PIÙ VELOCE DELLA RAM.

FORMULA TEMPO DI ACCESSO:

$$\bar{t}_{ACC} = \bar{t}_C + (1-h)\bar{t}_M$$

QUANDO TEMPO LA CPU CI METTE A RECUPERARE UN DATO. → $\bar{t}_C$ TEMPO DELLA CACHE
 HIT RATE (QUANTE VOLTE IL DATO ERA IN CACHE) → h TEMPO DELLA MEMORIA → $\bar{t}_M$

LOCALITÀ SPAZIALE: SE ACCEDO AD UN DATO (COME UN ARRAY) È MOLTO PROBABILE CHE HO BISOGNO DEI DATI CONTIGUI AD ESSO.

LOCALITÀ TEMPORALE: SE HO USATO UN DATO È MOLTO PROBABILE CHE MI SERVA DOPO (CONTATORE DI UN CICLO).

DEFINIZIONI:

- **BLOCCO**: PEZZO DI DATI CHE VIENE SPOSTATO DALLA RAM ALLA CACHE.
- **LINEA**: LO SLOT FISICO IN CUI INSERIRE I BLOCCHI.
- **RIGA**: L'INDICE → IN UNA CACHE DIRETTA LA RIGA HA 1 LINEA.
→ IN UNA CACHE A N-VIE LA RIGA HA N-LINEE.

3 TIPI DI CACHE:

- **CORRISPONDENZA DIRETTA**: LOGICA: OGNI BLOCCO HA 1 SOLO POSTO (1 LINEA)

FORMULE:

$$\text{OFFSET} = \log_2(\Delta IM \cdot B_{\text{locco}}) = \log_2(2^5) = 5 \text{ BIT}$$

$$\text{RIGA} = \log_2(\text{NUM. RIGHE TOT.}) = \log_2(2^{10}) = 10 \text{ BIT}$$

$$\text{TAG} = 32 - 10 - 5 = 17 \text{ BIT}$$

$$N_{\text{MBlocco}} = 32 \text{ byte}$$

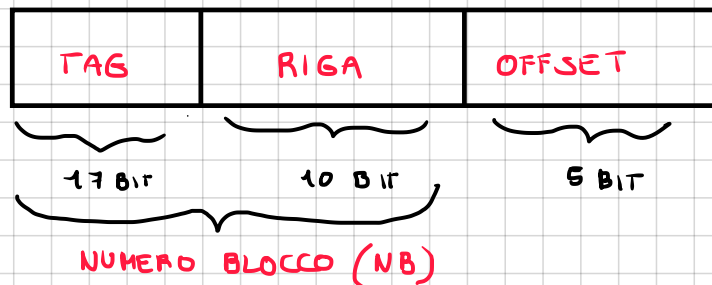
$$N_{\text{M.RigheTot.}} = 1024$$

TIPO CACHE DI 64 KB

$$\text{NUM. RIGHE TOT.} =$$

$$\frac{\text{CAPACITÀ DELLA CACHE}}{\text{DIM. BLOCCO}}$$

A =



32 BIT PERCHÉ
VOGHAMO INDIRIZZARE
TUTTA LA MEMORIA DI
UNA RAM A 4GB
(2^{32} BYTE)

INDIRIZZO
A 32 BIT

$$NB = A / \text{DIMBLOCCO}$$

oppure

$$A / \text{OFFSET}$$

... 1101101 | 10010

$$\rightarrow 2^k = 2^5$$

$$\text{TAG} = NB / \text{NUM. RIGHE TOT.}$$

$$\text{OFFSET} = A \% \text{DIM BLOCCO}$$

$$\text{RIGA} = NB \% \text{NUM. RIGHE TOT.}$$

DIFETTO: SE 2 INDIRIZZI FINISCONO NELLA STESSA RIGA, AVVIENE UN CONFLITTO, SI SOVRASCRIVONO A VICENZA.

• SET-ASSOCIATIVA A N-VIE:

LOGICA: LA CACHE È DIVISA SEMPRE IN RIGHE, MA OGNI RIGA HA N-POSTI (VIE) IN CUI OSPITARE N-BLOCCHI DELLA RAM.

FORMULE:

$$\text{NUM. RIGHE TOT.} =$$

$$\frac{\text{CAPACITÀ DELLA CACHE}}{\text{DIM. BLOCCO} \times \text{N. VIE}}$$

VANTAGGIO: RIDUCI I CONFLITTI, SE LA VIA 1 È OCCUPATA USI LA VIA 2.

• ASSOCIATIVA PURA:

LOGICA: NIENTE RIGA, IL BLOCCO PUÒ ANDARE OVUNQUE

COSTO: PER TROVARE IL BLOCCO DEVO CONFRONTARE TUTTI I TAG.

POLITICHE DI SCRITTURA:

– **WRITE-THROUGH:** SCRIVE SU CACHE E RAM CONTEMPORANEAMENTE

VANTAGGI: RAM E CACHE SEMPRE AGGIORNATI (SE SALTA LA CORRENTE RIMANGO = NO NELLA RAM).

SVANTAGGI: È LENTO, LA CPU DEVE ASPETTARE I TEMPI DELLA RAM.

– **WRITE-BACK:** È LA POLITICA USATA NEI PROCESSORI MODERNI, SCRIVI SOLO NELLA CACHE, LA CPU NON ASPETTA LA RAM, PERÒ IN QUESTO MODO LA CACHE È SEMPRE AGGIORNATA MA LA RAM CONTIENE DATI VECCHI.

DIRTY BIT: – QUANDO IL BLOCCO VIENE CARICATO DALLA RAM ALLA CACHE IL DIRTY BIT = 0.

– QUANDO LA CPU MODIFICA ANCHE UN SOLO BIT DEL BLOCCO IL DIRTY BIT = 1.

QUANDO LA CACHE È PIENA, SE DOBBIAMO SOVRASCRIVERE UN BLOCCO, BISOGNA CONTROL. IL DIRTY BIT, SE È ZERO ALLORA POSSIAMO SOVRASCRIVERLO, SE È UNO BISOGNA PRIMA COPIARLO IN RAM.

NELLA CACHE È PRESENTE UN ULTIMO FLAG OVVERO "VALID":

- SE $VALID = 0$, INDICA CHE IL POSTO È VUOTO/SPAZZATURA.
- SE $VALID = 1$, INDICA CHE QUI C'È UN DATO VERO.

L'OVERHEAD INDICA TUTTO LO SPAZIO DI MEMORIA "SPRECATO" PER GESTIRE LA CACHE:

$$OVERHEAD = 1 \text{ BIT (VALID)} + 1 \text{ BIT (DIRTY BIT)} + N \cdot \text{BIT (TAG)}$$

$$OVERHEAD \text{ PER TUTTI I BLOCCHI} = OVERHEAD \times NB \rightarrow \text{PER CONVERTIRE IN BYTE FACCIO DIVISO 8.}$$

$$\text{PERCENTUALE DI SPAZIO OCCUPATO DALL'OVERHEAD} = \frac{OVERHEAD \text{ TOTALE}}{OVERHEAD \text{ TOTALE} + CAPACITÀ \text{ CACHE}} \times 100$$

ESERCIZIO:

Contesto: Una CPU utilizza indirizzi a 32 bit. La memoria cache ha una capacità di 64 KB (dedicata ai soli dati) con blocchi da 32 byte. Si supponga di utilizzare una politica di scrittura Write-Back (è quindi necessario il Dirty Bit). La memoria RAM è di 4GB (2^{32} byte).

DATI:

$$CAPACITÀ \text{ DATI} : 64 \text{ KB} = 2^{10} \cdot 2^6 \text{ BYTE} = 2^{16} \text{ BYTE} = 65.536 \text{ BIT}$$

$$\text{DIM. BLOCCO} : 32 \text{ BYTE} = 2^5 \text{ BYTE}$$

$$\text{INDIRIZZO RAM} : 32 \text{ BIT}$$

$$\text{BIT DI OFFSET} : \log_2(2^5) = 5 \text{ BIT}$$

$$\text{NUM. BLOCCHI TOT} : 2^{16} / 2^5 = 2^{11} = 2048 \text{ BLOCCHI}$$

1. CACHE A CORRISPONDENZA DIRETTA:

$$\text{NUMERO RIGHE} = 2^{11} = 2048 \text{ RIGHE (1 X OGNI BLOCCO)}$$

$$\text{BIT RIGA} = 11 \text{ BIT}$$

$$\text{BIT OFFSET} = \log_2(2^5) = 5 \text{ BIT}$$

$$\text{BIT TAG} = 32 - 11 - 5 = 16 \text{ BIT}$$

$$\text{OVERHEAD PER BLOCCO} = 1 (\text{VALID}) + 1 (\text{DIRTY}) + 16 (\text{TAG}) = 18 \text{ BIT}$$

$$\text{OVERHEAD TOTALE} = 18 \times 2048 = 36.864 \text{ BIT} = 4.608 \text{ BYTE}$$

$$\text{OCCUPAZIONE \%} = (4608 / (65.536 + 4608)) \times 100 = 6,57 \%$$

2. CACHE SET ASSOCIATIVA A 2 VIE:

$$\text{NUMERO RIGHE} = 2^{11} / 2 = 2^{10} = 1024 \text{ RIGHE}$$

$$\text{BIT RIGA} = 10 \text{ BIT}$$

$$\text{BIT OFFSET} = 5 \text{ BIT}$$

$$\text{BIT TAG} = 32 - 10 - 5 = 17 \text{ BIT}$$

$$\text{OVERHEAD PER BLOCCO} = 1 (\text{VALID}) + 1 (\text{DIRTY}) + 17 (\text{TAG}) = 19 \text{ BIT}$$

$$\text{OVERHEAD TOTALE} = 19 \times 2048 = 38.912 \text{ BIT} = 4.864 \text{ BYTE}$$

3. CACHE ASSOCIATIVA PURA:

BIT INDEX = 0 BIT

BIT OFFSET = 5 BIT

BIT TAG = $32 - 5 = 27$ BIT

OVERHEAD PER BLOCCO = 1 (VALID) + 1 (DIRTY) + 27 (TAG) = 29 BIT

OVERHEAD TOTALE = $2048 \times 29 = 59.392$ BIT = 7.424 BYTE

OCCUPAZIONE % = $(7.424 / (7.424 + 65.536)) \times 100 = 10,18 \%$

LA CACHE PUÒ SERVIRE PER GESTIRE LE INFORMAZIONI NECESSARIE AD APPLICARE GLI ALGORITMI DI PREVISIONE DINAMICA DEI SALTI:

— LA CPU TIENE UNA PICCOLA PARTE DI MEMORIA DOVE SI ANNOTA LA STORIA DEI SALTI.

— AL POSTO DI UN DATO DELLA RAM TROVIAMO DEI BIT DI PREVISIONE (1 o 2)

LA CACHE PUÒ SERVIRE PER TRASFORMARE INDIRIZZI VIRTUALI IN INDIRIZZI FISICI. (TLB)

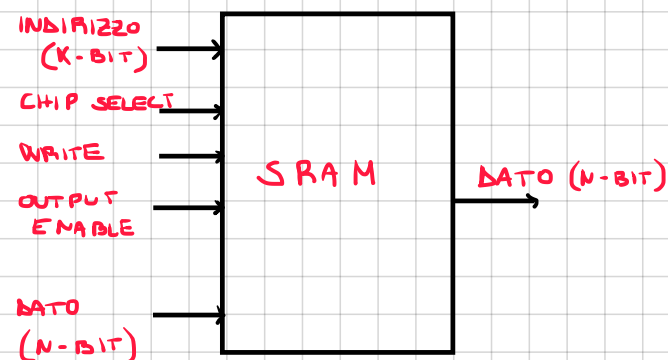
LA MEMORIA CACHE, È UNA MEMORIA VOLATILE, POSTA LOGICAMENTE TRA CPU E RAM, MOLTO VELOCE, NELL'ORDINE DEI NS, REALIZZATA CON MEMORIA SRAM, SERVE PER RENDERE DISPONIBILI I DATI ALLA CPU + VELOCEMENTE DELLA RAM.

SRAM (STATIC RANDOM ACCESS MEMORY): UTILIZZA 6 TRANSISTOR QUINDI È PIÙ COSTOSA

DI UNA DRAM MA PIÙ VELOCE PERCHÉ NON NECESSITA DI REFRESH.

UTILIZZA UNA STRUTTURA A MATRICE, AD ESEMPIO PER 4 MBIT (2^{22} BIT) DI SRAM X 8 BIT, UTILIZZANDO UN INDIRIZZO DA 22 BIT, DI CUI I PRIMI 12 BIT HANNO SIGNIFICATIVI SERVONO PER INDIRIZZARE LA RIGA E GLI ULTIMI 10 BIT PER LA COLONNA PER OGNI BLOCCO DA $4K \times 1024$ BIT.

LA RIGA VIENE INDIRIZZATA DA UN DECODER 12 TO 4096, MENTRE PER TUTTI E 8 I BLOCCHI DA $4K \times 1024$ VENGONO USATI UN MUX PER OGNUNO QUINDI 8 MUX TOTALI PER SELEZIONARE LA COLONNA E DARE UN BYTE COME OUTPUT.



CHIP SELECT: IN CASO DI PIÙ CHIP DI MEMORIA NE SEGLIE UNA.

WRITE: QUANDO A 1 ABILITA LA SCRITTURA (OUTPUT = 0)

OUTPUT: QUANDO A 1 ABILITA LA LETTURA (WRITE = 0).

DATO: SE WRITE A 1 È IL DATO DA SCRIVERE NELLA MEMORIA.

MEMORIA CENTRALE (RAM):

RANDOM ACCESS MEMORY, MEMORIA VOLATILE, POSTA LOGICAMENTE TRA **CACHE** E **MEMORIA PERMANENTE**, CONTIENE I DATI DEI PROGRAMMI, ED È COSTRUITA USANDO LA TECNOLOGIA DI **DRAM**.

OGNI VOLTA

DRAM (DYNAMIC RANDOM ACCESS MEMORY): SONO **DINAMICHE** PERCHÉ I DATI DEVONO ESSERE

REFRESHATI PERCHÉ MEMORIZZATI IN UN **CONDENSATORE** CHE SI SCARICA DOPO ALCUNI **mn**.

VANTAGGIO: MENO COSTOSO (USA 1 SOLO **TRANSISTOR**) **SVANTAGGIO**: MENO VELOCE (**REFRESH**)

UTILIZZA UN SISTEMA A **MATRICE**, AD ESEMPIO PER INDIRIZZARE 4 MBIT (2^{22} BIT) SI UTILIZZA UN INDIRIZZO DA 11 BIT (11 PER LA RIGA CON **RAS** ATTIVO E 11 PER LA COLONNA CON **CAS** ATTIVO).

↳ **ROW ADDRESS STROBE**

PER LA RIGA USO UN **DECODER** MENTRE PER LA COLONNA UN **MUX**.

PER POTER LEGGERE UN **BYTE** DEVO METTERE 8 CHIP DI MEMORIA (PERCHÉ OGNUNO NE HA 1 BIT).

LE **SDRAM** EFFETTUANO ACCESSI **SINCRONIZZATI CON IL CLOCK** (SONO PIÙ VELOCI PERCHÉ LA CPU NON DEVE ASPETTARE UNA RISPOSTA ASINCRONA).

LE **SDRAM DDR (DOUBLE DATA RATE)** ONVERO EFFETTUANO IL DOPIO DEI TRASFERIMENTI PERCHÉ EFFETTUANO GLI ACCESSI SUI **FRONTI DI SALITA E DISCESA** DEL CLOCK.

DISCHI MAGNETICI:

I DATI SONO SCRITTI SU **PIATTI**, DIVISI IN **TRACCE CONCENTRICHE**, SU CUI LEGGONO E SCRIVONO **TESTINE** PER OGNI SUPERFICIE DEL PIATTO.

TEMPO DI ACCESSO: **SEEK** + MEZZA ROTAZIONE + TRASFERIMENTO DATI

QUINDI LE TESTINE SI POSIZIONANO SULLA TRACCIA GIUSTA (**SEEK**) DOPO DI CHE RUOTANDO SI POSIZIONANO SUL GIUSTO SETTORE E POI TRASFERISCONO I DATI.

CONTROLLER:

I DISPOSITIVI DI **INPUT / OUTPUT** SONO COLLEGATI AL BUS TRAMITE UN'INTERFACCIA CHIAMATO **CONTROLLER**, IN GENERE LA CPU PER COMUNICARE CON QUESTI DISPOSITIVI USA IL CONTROLLER.

CONTROLLER DMA: **DIRECT MEMORY ACCESS**

PERMETTE DI TRASFERIRE I DATI DALL' **HARD DISK** ALLA **RAM** SENZA "BLOCCARE" LA **CPU**, ONVERO SPOSTA I DATI AL POSTO SUO E NEL MENTRE PUÒ TORNARE A FARE I CALCOLI.

INTERFACCIE:

DI ARCHIVIAZIONE

SONO I CONNETTORI FISICI PER I DISPOSITIVI, LE PRIME VERSIONI FURONO **UE** CHE USAVANO IL VECCHIO **INDIRIZZAMENTO FISICO**, PER POI EVOLVERSI NEL TEMPO INTRODUCENDO L'**LBA (COORDINATE LOGICHE PARTENDO DA 0 FINO A UN NUMERO N)** FINO AD ARRIVARE ALLE INTERFACCIE **SCSI** CHE POSSONO COLLEGARE **7 DISPOSITIVI IN PARALLELO** E RAGGIUNGERE **640 MB/S** DI TRASFERIMENTO.

TUTT'OGGI IL METODO USATO DALLO STANDARD MODERNO **SSD NVMe** È QUELLO DI COLLEGARSI DIRETTAMENTE SUL **PCI EXPRESS**.

• GARANTIRE LA SICUREZZA NEL SISTEMA:

CON IL SERVIZIO **RAID** (REDUNDANT ARRAY OF INDEPENDENT DISK) L'OBIETTIVO È QUELLO DI **PARALLELIZZARE**.

- **RAID 0** = IL DATO VIENE **DISTRIBUITO** IN TUTTI I DISCHI, UTILIZZA TUTTO LO SPAZIO ED È **VELOCISSIMA**, MA TANTO **INAFFIDABILE** (PERDO UN DISCO E PERDO TUTTO).
- **RAID 1** = IL DATO VIENE **SPECCHIATO** IN TUTTI I DISCHI, UTILIZZA LA METÀ DELLO SPAZIO, MA È MOLTO **AFFIDABILE**.
- **RAID 2** = DIVIDE I DATI IN **SINGOLI BIT** SU OGNI DISCO E USA IL **CODICE DI HAMMING** PER LA CORREZIONE DEGLI ERRORI, BISOGNA USARE TROPPI DISCHI SE IL DATO È LUNGO, ED È OBSOLETO PERCHÉ I DISCHI HANNO GIÀ INTEGRATA LA CORREZIONE DEGLI ERRORI (**ECC**).
- **RAID 3** = DIVIDE I DATI IN **SINGOLI BYTE** E USA IL BIT DI **PARITÀ**.
- **RAID 4** = COME IL RAID 3 HA DIVIDE IL DATO IN **BLOCCHI** E UN DISCO DEDICATO PER LA **PARITÀ**.
- **RAID 5** = DATI SUDDIVISI A BLOCCHI, MA IL **BIT DI PARITÀ** VIENE DISTRIBUITO SU TUTTI I DISCHI.

ECC:

L'**ECC** (ERROR CORRECTING CODE) SERVE PER CORREGGERE IL FENOMENO DEL **BIT FLIP** SULLE MEMORIE RAM E DISPOSITIVI DI ARCHIVIAZIONE, DOVUTO A **FENOMENI FISICI**.

RISERVA DEI BIT PER L'ECC COME PER I DISCHI È UNA VARIANTE DEL **CODICE DI HAMMING** PER LA RAM IN MODO TALE DA NON PERMETTERE AI DATI DI **CORROMPERSI** O DI CAMBIARE VALORE IN MODO NON VOLUTO.

COSA FA LA CPU?

È IL CERVELLO DI UN CALCOLATORE ED OPERA SU **REGISTRI, CACHE E RAM**.

È IN GRADO DI COMPIERE OPERAZIONI ESTREMAMENTE **SEMPLICI** COME COPIARE DATI DA UNA CELLA ALL'ALTRA O SOMMARE IL CONTENUTO DI 2 REGISTRI.